INCIDENT & REMEDIATION REPORT

RESOLVED

Event Sync Reliability

Automated nightly event-listing synchronization

Prepared 30 June 2026 / GitHub Actions workflow "Sync event listings"

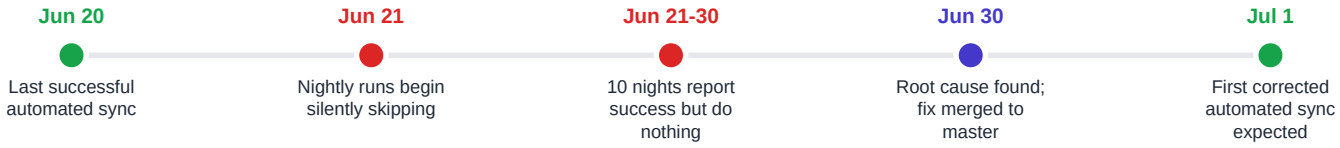
Executive Summary

The website's nightly event-listing synchronization silently stopped doing useful work on **21 June 2026**. The automated job continued to run every night and continued to report **success**, but it was skipping the actual synchronization step every time. As a result, no event data was automatically refreshed for roughly ten consecutive nights. The listings on the site stayed current only because of occasional manual runs.

The failure came from two things acting together. GitHub's scheduled jobs run on a best-effort basis and were firing several hours later than requested, and the job contained a guard that would only proceed if the local clock read **exactly** 9:00 PM Pacific at the moment it ran. Because the job kept arriving after midnight Pacific, the guard rejected every run. The root cause was identified and a redesigned, far more resilient workflow was built, reviewed, merged to `master`, and is now live.

> **Impact at a glance**
>
> Last automated sync: **20 June 2026** | Nights skipped: **~10** | Data loss: **none** (sources unchanged; full sync on resume) | Site outage: **none** | Status: **Fixed and deployed**

Timeline



What the System Was Supposed to Do

A GitHub Actions workflow (`.github/workflows/event-sync.yml`) is scheduled to run once a night at about 9:00 PM Pacific. On each run it pulls the latest events from Purplepass and Facebook, regenerates the event content files, verifies the site still builds, and, if anything changed, commits the results back to the repository. A small bookkeeping file, `event-sync/state.json`, records the timestamp of the last run.

Because GitHub's scheduler works in UTC and Pacific time shifts by an hour across daylight saving, the original design scheduled the job at two UTC times and then used a guard step to let only the 9:00 PM Pacific instance proceed.

The Defect

The original guard, in plain terms, said: *"only run the sync if the Pacific hour is exactly 21."* In code:

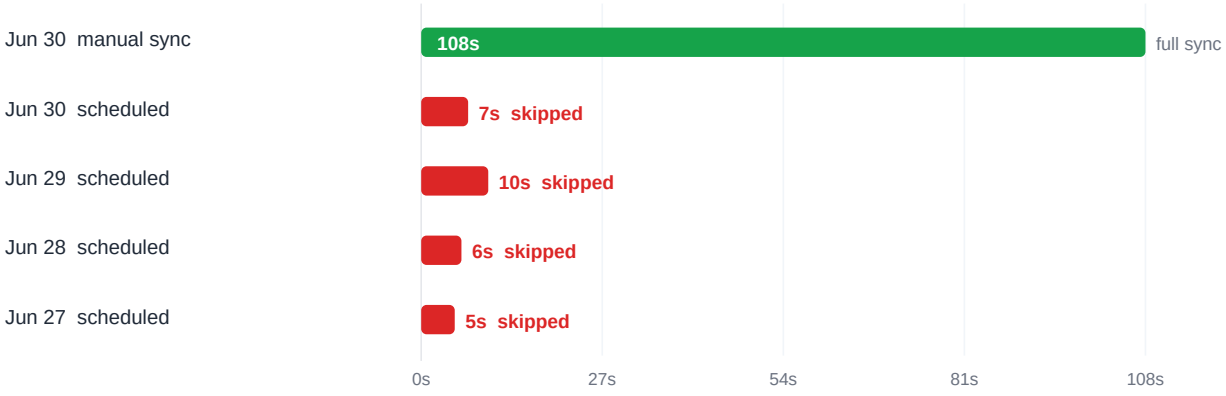
```
HOUR="$(TZ=America/Los_Angeles date +%H)"
if [ "$HOUR" = "21" ]; then       # must be EXACTLY 9 PM Pacific
  echo "run=true"  >> "$GITHUB_OUTPUT"
else
  echo "run=false" >> "$GITHUB_OUTPUT"   # otherwise skip everything
fi
```

This guard assumes the job actually starts at the requested time. It does not. GitHub Actions explicitly documents that scheduled runs are best-effort and may be delayed during periods of high load. In practice the runs were arriving **3 to 5 hours late**. By the time the job executed, the Pacific clock read midnight to 2:00 AM, never 21, so the guard set `run=false` and skipped every subsequent step.

> **Why nobody got an error**
>
> The guard step itself completed successfully, and all the real work was marked as conditionally skipped. A run that does nothing therefore still finishes **green**. The only visible tell was runtime: a real sync takes about 1m48s, while the skipped runs finished in 5 to 10 seconds.

Evidence: Run Durations

The five most recent runs make the pattern obvious. The single long bar is a real (manually triggered) sync; every scheduled run collapsed to a few seconds because it bailed at the guard.



Selected scheduled runs: requested vs. actual fire time

Date	Requested (UTC)	Actual fire (UTC)	Pacific clock	Result
Jun 30	04:00	07:49	12:49 AM	skipped
Jun 29	04:00	08:53	01:53 AM	skipped
Jun 27	04:00	06:49	11:49 PM	skipped

Jun 22	04:00	09:44	02:44 AM	**skipped**
Jun 21	04:00	08:24	01:24 AM	**skipped**

Requested 04:00 UTC corresponds to 9:00 PM Pacific. Every run arrived hours later, after the guard's window had passed.

The Remedy

Rather than patch the exact-time check, the workflow was redesigned around a more robust principle: **stop depending on precisely when the scheduler fires**. Two changes work together.

1. Redundant triggers

The job is now scheduled to fire once an hour across the whole late-evening window instead of at a single instant. If GitHub delays or drops the first trigger, a later one still lands the same night.

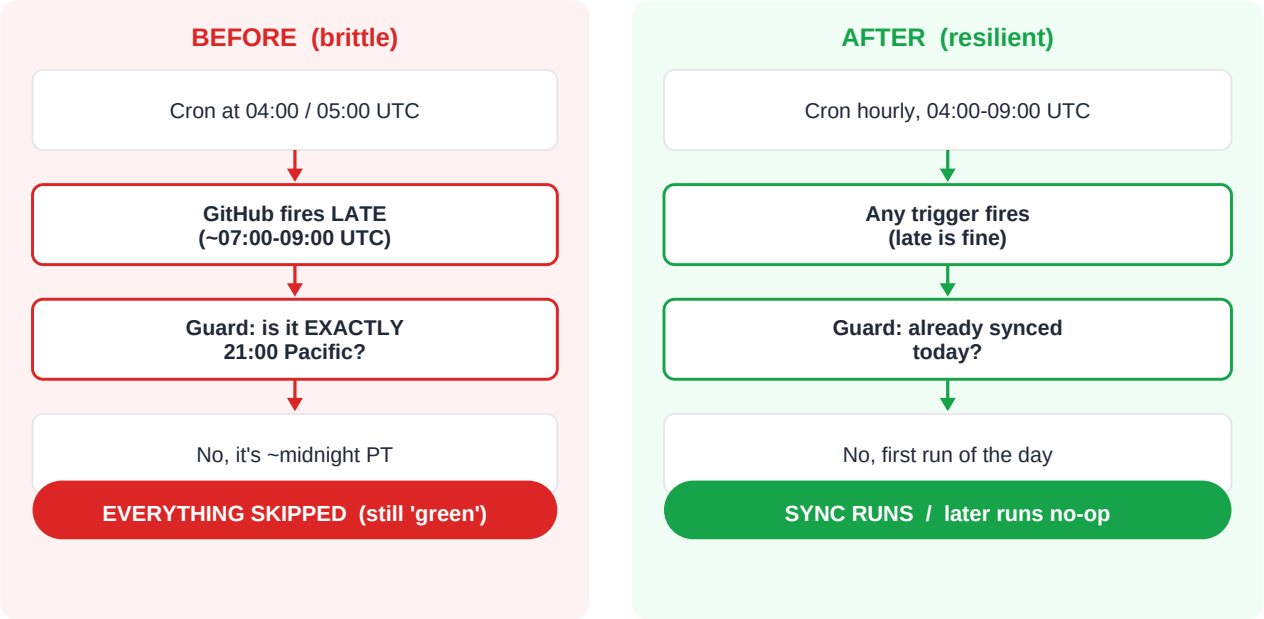
```
schedule:
  - cron: "7 4-9 * * *"   # hourly, 9 PM to 2 AM Pacific window
```

2. Idempotent, self-deciding job

Each run now decides for itself whether a sync is actually due by reading the last-run timestamp from `state.json` and comparing its Pacific date to today. A real sync happens **at most once per Pacific day**; whichever trigger fires first does the work, and the remaining triggers that night become cheap no-ops. The decision no longer depends on the wall-clock time at the moment of execution.

```
today="$(TZ=America/Los_Angeles date +%F)"
last="$(jq -r '.lastRun.at // empty' event-sync/state.json)"
last_day="$(TZ=America/Los_Angeles date -d "$last" +%F)"
if [ "$last_day" = "$today" ]; then
  run=false      # already synced today -> skip (intentional no-op)
else
  run=true       # not yet synced today -> run
fi
```

Before vs. After



Why the New Design Is Robust

- **Tolerant of late runs.** The job no longer cares what time it actually starts; a delayed trigger still performs the sync.
- **Tolerant of dropped runs.** Six triggers per night provide redundancy, so a missed schedule does not cost a day of updates.
- **No duplicate work.** The once-per-day check plus the existing concurrency lock guarantee exactly one real sync per day even if several triggers overlap.
- **Self-correcting.** If a whole night is missed, the next successful trigger simply catches up, because the decision is based on the last actual sync, not the calendar.
- **Truthful status.** A skipped run is now an intentional, logged 'already synced today' no-op rather than a silent failure dressed up as success.

Verification

The fix was committed on a dedicated branch, opened as pull request **#5**, and merged to `master` (commit `b3583b7`) so it governs the next scheduled run. Three loose, single-use diagnostic scripts left over from earlier image investigations were removed in the same cleanup. A follow-up check is scheduled for the morning of **1 July 2026** to confirm the overnight run behaved as intended:

- Exactly one scheduled run does real work (about one to two minutes) and commits any changes.
- Later scheduled triggers that night finish in seconds via the 'already synced today' path.
- `state.json`'s last-run timestamp shows a fresh date with trigger type `schedule`.

Bottom line

The site never went down and no data was lost. The automated refresh was silently idle for about ten nights because a brittle timing check collided with GitHub's best-effort scheduler. It has been replaced with a design that succeeds regardless of exactly when, or how reliably, the scheduler fires.

Appendix: Deployed Workflow (key sections)

Trigger and decision logic now in production on `master`:

```
on:
  schedule:
    - cron: "7 4-9 * * *"          # hourly across the evening window
  workflow_dispatch:               # manual runs always sync

jobs:
  sync:
    steps:
      - uses: actions/checkout@v4

      - name: Decide whether to sync
        id: decide
        run: |
          if [ "$GITHUB_EVENT_NAME" = "workflow_dispatch" ]; then
            echo "run=true" >> "$GITHUB_OUTPUT"; exit 0
          fi
          today="$(TZ=America/Los_Angeles date +%F)"
          last="$(jq -r '.lastRun.at // empty' event-sync/state.json)"
          last_day="$(TZ=America/Los_Angeles date -d "$last" +%F)"
          if [ "$last_day" = "$today" ]; then
            echo "run=false" >> "$GITHUB_OUTPUT"   # already synced today
          else
            echo "run=true"  >> "$GITHUB_OUTPUT"
          fi

      # subsequent steps gated on: steps.decide.outputs.run == 'true'
      #   - Set up Node / npm ci
      #   - npm run events:sync   (Purplepass + Facebook)
      #   - npm run build         (verify site)
      #   - commit & push synced files + state.json
```

Reference

File: `.github/workflows/event-sync.yml` / PR #5 / merge commit `b3583b7` / state file: `event-sync/state.json`